

# **xv6: a simple, Unix-like teaching operating system**

Russ Cox

Frans Kaashoek

Robert Morris

August 31, 2024



# Contents

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Operating system interfaces</b>                              | <b>9</b>  |
| 1.1      | Processes and memory . . . . .                                  | 10        |
| 1.2      | I/O and File descriptors . . . . .                              | 13        |
| 1.3      | Pipes . . . . .                                                 | 16        |
| 1.4      | File system . . . . .                                           | 17        |
| 1.5      | Real world . . . . .                                            | 19        |
| 1.6      | Exercises . . . . .                                             | 19        |
| <br>     |                                                                 |           |
| <b>2</b> | <b>Operating system organization</b>                            | <b>21</b> |
| 2.1      | Abstracting physical resources . . . . .                        | 22        |
| 2.2      | User mode, supervisor mode, and system calls . . . . .          | 22        |
| 2.3      | Kernel organization . . . . .                                   | 23        |
| 2.4      | Code: xv6 organization . . . . .                                | 25        |
| 2.5      | Process overview . . . . .                                      | 26        |
| 2.6      | Code: starting xv6, the first process and system call . . . . . | 28        |
| 2.7      | Security Model . . . . .                                        | 29        |
| 2.8      | Real world . . . . .                                            | 29        |
| 2.9      | Exercises . . . . .                                             | 30        |
| <br>     |                                                                 |           |
| <b>3</b> | <b>Page tables</b>                                              | <b>31</b> |
| 3.1      | Paging hardware . . . . .                                       | 31        |
| 3.2      | Kernel address space . . . . .                                  | 34        |
| 3.3      | Code: creating an address space . . . . .                       | 35        |
| 3.4      | Physical memory allocation . . . . .                            | 37        |
| 3.5      | Code: Physical memory allocator . . . . .                       | 37        |
| 3.6      | Process address space . . . . .                                 | 38        |
| 3.7      | Code: sbrk . . . . .                                            | 39        |
| 3.8      | Code: exec . . . . .                                            | 40        |
| 3.9      | Real world . . . . .                                            | 41        |
| 3.10     | Exercises . . . . .                                             | 42        |

|          |                                           |           |
|----------|-------------------------------------------|-----------|
| <b>4</b> | <b>Traps and system calls</b>             | <b>43</b> |
| 4.1      | RISC-V trap machinery . . . . .           | 44        |
| 4.2      | Traps from user space . . . . .           | 45        |
| 4.3      | Code: Calling system calls . . . . .      | 47        |
| 4.4      | Code: System call arguments . . . . .     | 47        |
| 4.5      | Traps from kernel space . . . . .         | 48        |
| 4.6      | Page-fault exceptions . . . . .           | 48        |
| 4.7      | Real world . . . . .                      | 51        |
| 4.8      | Exercises . . . . .                       | 51        |
| <b>5</b> | <b>Interrupts and device drivers</b>      | <b>53</b> |
| 5.1      | Code: Console input . . . . .             | 53        |
| 5.2      | Code: Console output . . . . .            | 54        |
| 5.3      | Concurrency in drivers . . . . .          | 55        |
| 5.4      | Timer interrupts . . . . .                | 55        |
| 5.5      | Real world . . . . .                      | 56        |
| 5.6      | Exercises . . . . .                       | 57        |
| <b>6</b> | <b>Locking</b>                            | <b>59</b> |
| 6.1      | Races . . . . .                           | 60        |
| 6.2      | Code: Locks . . . . .                     | 63        |
| 6.3      | Code: Using locks . . . . .               | 64        |
| 6.4      | Deadlock and lock ordering . . . . .      | 64        |
| 6.5      | Re-entrant locks . . . . .                | 66        |
| 6.6      | Locks and interrupt handlers . . . . .    | 67        |
| 6.7      | Instruction and memory ordering . . . . . | 67        |
| 6.8      | Sleep locks . . . . .                     | 68        |
| 6.9      | Real world . . . . .                      | 69        |
| 6.10     | Exercises . . . . .                       | 69        |
| <b>7</b> | <b>Scheduling</b>                         | <b>71</b> |
| 7.1      | Multiplexing . . . . .                    | 71        |
| 7.2      | Code: Context switching . . . . .         | 72        |
| 7.3      | Code: Scheduling . . . . .                | 73        |
| 7.4      | Code: mycpu and myproc . . . . .          | 74        |
| 7.5      | Sleep and wakeup . . . . .                | 75        |
| 7.6      | Code: Sleep and wakeup . . . . .          | 78        |
| 7.7      | Code: Pipes . . . . .                     | 79        |
| 7.8      | Code: Wait, exit, and kill . . . . .      | 79        |
| 7.9      | Process Locking . . . . .                 | 81        |
| 7.10     | Real world . . . . .                      | 82        |
| 7.11     | Exercises . . . . .                       | 83        |

|           |                                 |            |
|-----------|---------------------------------|------------|
| <b>8</b>  | <b>File system</b>              | <b>85</b>  |
| 8.1       | Overview . . . . .              | 85         |
| 8.2       | Buffer cache layer . . . . .    | 87         |
| 8.3       | Code: Buffer cache . . . . .    | 87         |
| 8.4       | Logging layer . . . . .         | 88         |
| 8.5       | Log design . . . . .            | 89         |
| 8.6       | Code: logging . . . . .         | 90         |
| 8.7       | Code: Block allocator . . . . . | 91         |
| 8.8       | Inode layer . . . . .           | 92         |
| 8.9       | Code: Inodes . . . . .          | 93         |
| 8.10      | Code: Inode content . . . . .   | 94         |
| 8.11      | Code: directory layer . . . . . | 95         |
| 8.12      | Code: Path names . . . . .      | 96         |
| 8.13      | File descriptor layer . . . . . | 97         |
| 8.14      | Code: System calls . . . . .    | 98         |
| 8.15      | Real world . . . . .            | 99         |
| 8.16      | Exercises . . . . .             | 100        |
| <b>9</b>  | <b>Concurrency revisited</b>    | <b>103</b> |
| 9.1       | Locking patterns . . . . .      | 103        |
| 9.2       | Lock-like patterns . . . . .    | 104        |
| 9.3       | No locks at all . . . . .       | 105        |
| 9.4       | Parallelism . . . . .           | 105        |
| 9.5       | Exercises . . . . .             | 106        |
| <b>10</b> | <b>Summary</b>                  | <b>107</b> |



# Chapter 1

## Operating system interfaces

The job of an operating system is to share a computer among multiple programs and to provide a more useful set of services than the hardware alone supports. An operating system manages and abstracts the low-level hardware, so that, for example, a word processor need not concern itself with which type of disk hardware is being used. An operating system shares the hardware among multiple programs so that they run (or appear to run) at the same time. Finally, operating systems provide controlled ways for programs to interact, so that they can share data or work together.

An operating system provides services to user programs through an interface. Designing a good interface turns out to be difficult. On the one hand, we would like the interface to be simple and narrow because that makes it easier to get the implementation right. On the other hand, we may be tempted to offer many sophisticated features to applications. The trick in resolving this tension is to design interfaces that rely on a few mechanisms that can be combined to provide much generality.

This book uses a single operating system as a concrete example to illustrate operating system concepts. That operating system, xv6, provides the basic interfaces introduced by Ken Thompson and Dennis Ritchie’s Unix operating system [17], as well as mimicking Unix’s internal design. Unix provides a narrow interface whose mechanisms combine well, offering a surprising degree of generality. This interface has been so successful that modern operating systems—BSD, Linux, macOS, Solaris, and even, to a lesser extent, Microsoft Windows—have Unix-like interfaces. Understanding xv6 is a good start toward understanding any of these systems and many others.

As Figure 1.1 shows, xv6 takes the traditional form of a *kernel*, a special program that provides services to running programs. Each running program, called a *process*, has memory containing instructions, data, and a stack. The instructions implement the program’s computation. The data are the variables on which the computation acts. The stack organizes the program’s procedure calls. A given computer typically has many processes but only a single kernel.

When a process needs to invoke a kernel service, it invokes a *system call*, one of the calls in the operating system’s interface. The system call enters the kernel; the kernel performs the service and returns. Thus a process alternates between executing in *user space* and *kernel space*.

As described in detail in subsequent chapters, the kernel uses the hardware protection mechanisms provided by a CPU<sup>1</sup> to ensure that each process executing in user space can access only

---

<sup>1</sup>This text generally refers to the hardware element that executes a computation with the term *CPU*, an acronym

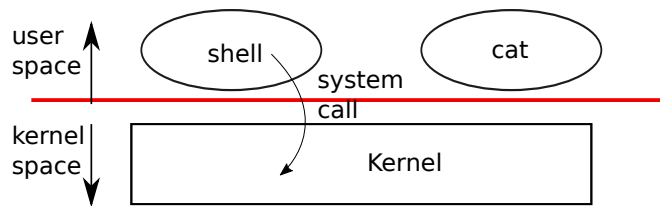


Figure 1.1: A kernel and two user processes.

its own memory. The kernel executes with the hardware privileges required to implement these protections; user programs execute without those privileges. When a user program invokes a system call, the hardware raises the privilege level and starts executing a pre-arranged function in the kernel.

The collection of system calls that a kernel provides is the interface that user programs see. The xv6 kernel provides a subset of the services and system calls that Unix kernels traditionally offer. Figure 1.2 lists all of xv6’s system calls.

The rest of this chapter outlines xv6’s services—processes, memory, file descriptors, pipes, and a file system—and illustrates them with code snippets and discussions of how the *shell*, Unix’s command-line user interface, uses them. The shell’s use of system calls illustrates how carefully they have been designed.

The shell is an ordinary program that reads commands from the user and executes them. The fact that the shell is a user program, and not part of the kernel, illustrates the power of the system call interface: there is nothing special about the shell. It also means that the shell is easy to replace; as a result, modern Unix systems have a variety of shells to choose from, each with its own user interface and scripting features. The xv6 shell is a simple implementation of the essence of the Unix Bourne shell. Its implementation can be found at (user/sh.c:1).

## 1.1 Processes and memory

An xv6 process consists of user-space memory (instructions, data, and stack) and per-process state private to the kernel. Xv6 *time-shares* processes: it transparently switches the available CPUs among the set of processes waiting to execute. When a process is not executing, xv6 saves the process’s CPU registers, restoring them when it next runs the process. The kernel associates a process identifier, or `PID`, with each process.

A process may create a new process using the `fork` system call. `fork` gives the new process an exact copy of the calling process’s memory: it copies the instructions, data, and stack of the calling process into the new process’s memory. `fork` returns in both the original and new processes. In the original process, `fork` returns the new process’s `PID`. In the new process, `fork` returns zero. The original and new processes are often called the *parent* and *child*.

---

for central processing unit. Other documentation (e.g., the RISC-V specification) also uses the words processor, core, and hart instead of CPU.



| System call                                      | Description                                                                                                    |
|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <code>int fork()</code>                          | Create a process, return child's PID.                                                                          |
| <code>int exit(int status)</code>                | Terminate the current process; status reported to <code>wait()</code> . No return.                             |
| <code>int wait(int *status)</code>               | Wait for a child to exit; exit status in <code>*status</code> ; returns child PID.                             |
| <code>int kill(int pid)</code>                   | Terminate process PID. Returns 0, or -1 for error.                                                             |
| <code>int getpid()</code>                        | Return the current process's PID.                                                                              |
| <code>int sleep(int n)</code>                    | Pause for <code>n</code> clock ticks.                                                                          |
| <code>int exec(char *file, char *argv[])</code>  | Load a file and execute it with arguments; only returns if error.                                              |
| <code>char *sbrk(int n)</code>                   | Grow process's memory by <code>n</code> zero bytes. Returns start of new memory.                               |
| <code>int open(char *file, int flags)</code>     | Open a file; flags indicate read/write; returns an fd (file descriptor).                                       |
| <code>int write(int fd, char *buf, int n)</code> | Write <code>n</code> bytes from <code>buf</code> to file descriptor <code>fd</code> ; returns <code>n</code> . |
| <code>int read(int fd, char *buf, int n)</code>  | Read <code>n</code> bytes into <code>buf</code> ; returns number read; or 0 if end of file.                    |
| <code>int close(int fd)</code>                   | Release open file <code>fd</code> .                                                                            |
| <code>int dup(int fd)</code>                     | Return a new file descriptor referring to the same file as <code>fd</code> .                                   |
| <code>int pipe(int p[])</code>                   | Create a pipe, put read/write file descriptors in <code>p[0]</code> and <code>p[1]</code> .                    |
| <code>int chdir(char *dir)</code>                | Change the current directory.                                                                                  |
| <code>int mkdir(char *dir)</code>                | Create a new directory.                                                                                        |
| <code>int mknod(char *file, int, int)</code>     | Create a device file.                                                                                          |
| <code>int fstat(int fd, struct stat *st)</code>  | Place info about an open file into <code>*st</code> .                                                          |
| <code>int link(char *file1, char *file2)</code>  | Create another name ( <code>file2</code> ) for the file <code>file1</code> .                                   |
| <code>int unlink(char *file)</code>              | Remove a file.                                                                                                 |

Figure 1.2: Xv6 system calls. If not otherwise stated, these calls return 0 for no error, and -1 if there's an error.

For example, consider the following program fragment written in the C programming language [7]:

```
int pid = fork();
if(pid > 0){
    printf("parent: child=%d\n", pid);
    pid = wait((int *) 0);
    printf("child %d is done\n", pid);
} else if(pid == 0){
    printf("child: exiting\n");
    exit(0);
} else {
    printf("fork error\n");
}
```

The `exit` system call causes the calling process to stop executing and to release resources such as memory and open files. Exit takes an integer status argument, conventionally 0 to indicate success and 1 to indicate failure. The `wait` system call returns the PID of an exited (or killed) child of the current process and copies the exit status of the child to the address passed to `wait`; if none of

the caller's children has exited, `wait` waits for one to do so. If the caller has no children, `wait` immediately returns `-1`. If the parent doesn't care about the exit status of a child, it can pass a `0` address to `wait`.

In the example, the output lines

```
parent: child=1234
child: exiting
```

might come out in either order (or even intermixed), depending on whether the parent or child gets to its `printf` call first. After the child exits, the parent's `wait` returns, causing the parent to print

```
parent: child 1234 is done
```

Although the child has the same memory contents as the parent initially, the parent and child are executing with separate memory and separate registers: changing a variable in one does not affect the other. For example, when the return value of `wait` is stored into `pid` in the parent process, it doesn't change the variable `pid` in the child. The value of `pid` in the child will still be zero.

The `exec` system call replaces the calling process's memory with a new memory image loaded from a file stored in the file system. The file must have a particular format, which specifies which part of the file holds instructions, which part is data, at which instruction to start, etc. Xv6 uses the ELF format, which Chapter 3 discusses in more detail. Usually the file is the result of compiling a program's source code. When `exec` succeeds, it does not return to the calling program; instead, the instructions loaded from the file start executing at the entry point declared in the ELF header. `exec` takes two arguments: the name of the file containing the executable and an array of string arguments. For example:

```
char *argv[3];

argv[0] = "echo";
argv[1] = "hello";
argv[2] = 0;
exec("/bin/echo", argv);
printf("exec error\n");
```

This fragment replaces the calling program with an instance of the program `/bin/echo` running with the argument list `echo hello`. Most programs ignore the first element of the argument array, which is conventionally the name of the program.

The xv6 shell uses the above calls to run programs on behalf of users. The main structure of the shell is simple; see `main` (`user/sh.c:146`). The main loop reads a line of input from the user with `getcmd`. Then it calls `fork`, which creates a copy of the shell process. The parent calls `wait`, while the child runs the command. For example, if the user had typed “`echo hello`” to the shell, `runcmd` would have been called with “`echo hello`” as the argument. `runcmd` (`user/sh.c:55`) runs the actual command. For “`echo hello`”, it would call `exec` (`user/sh.c:79`). If `exec` succeeds then the child will execute instructions from `echo` instead of `runcmd`. At some point `echo` will call `exit`, which will cause the parent to return from `wait` in `main` (`user/sh.c:146`).

You might wonder why `fork` and `exec` are not combined in a single call; we will see later that the shell exploits the separation in its implementation of I/O redirection. To avoid the wastefulness

of creating a duplicate process and then immediately replacing it (with `exec`), operating kernels optimize the implementation of `fork` for this use case by using virtual memory techniques such as copy-on-write (see Section 4.6).

Xv6 allocates most user-space memory implicitly: `fork` allocates the memory required for the child's copy of the parent's memory, and `exec` allocates enough memory to hold the executable file. A process that needs more memory at run-time (perhaps for `malloc`) can call `sbrk(n)` to grow its data memory by `n` zero bytes; `sbrk` returns the location of the new memory.

## 1.2 I/O and File descriptors

A *file descriptor* is a small integer representing a kernel-managed object that a process may read from or write to. A process may obtain a file descriptor by opening a file, directory, or device, or by creating a pipe, or by duplicating an existing descriptor. For simplicity we'll often refer to the object a file descriptor refers to as a "file"; the file descriptor interface abstracts away the differences between files, pipes, and devices, making them all look like streams of bytes. We'll refer to input and output as *I/O*.

Internally, the xv6 kernel uses the file descriptor as an index into a per-process table, so that every process has a private space of file descriptors starting at zero. By convention, a process reads from file descriptor 0 (standard input), writes output to file descriptor 1 (standard output), and writes error messages to file descriptor 2 (standard error). As we will see, the shell exploits the convention to implement I/O redirection and pipelines. The shell ensures that it always has three file descriptors open (user/sh.c:152), which are by default file descriptors for the console.

The `read` and `write` system calls read bytes from and write bytes to open files named by file descriptors. The call `read(fd, buf, n)` reads at most `n` bytes from the file descriptor `fd`, copies them into `buf`, and returns the number of bytes read. Each file descriptor that refers to a file has an offset associated with it. `read` reads data from the current file offset and then advances that offset by the number of bytes read: a subsequent `read` will return the bytes following the ones returned by the first `read`. When there are no more bytes to read, `read` returns zero to indicate the end of the file.

The call `write(fd, buf, n)` writes `n` bytes from `buf` to the file descriptor `fd` and returns the number of bytes written. Fewer than `n` bytes are written only when an error occurs. Like `read`, `write` writes data at the current file offset and then advances that offset by the number of bytes written: each `write` picks up where the previous one left off.

The following program fragment (which forms the essence of the program `cat`) copies data from its standard input to its standard output. If an error occurs, it writes a message to the standard error.

```
char buf[512];
int n;

for(;;){
    n = read(0, buf, sizeof buf);
    if(n == 0)
```

```

        break;
    if(n < 0){
        fprintf(2, "read error\n");
        exit(1);
    }
    if(write(1, buf, n) != n){
        fprintf(2, "write error\n");
        exit(1);
    }
}

```

The important thing to note in the code fragment is that `cat` doesn't know whether it is reading from a file, console, or a pipe. Similarly `cat` doesn't know whether it is printing to a console, a file, or whatever. The use of file descriptors and the convention that file descriptor 0 is input and file descriptor 1 is output allows a simple implementation of `cat`.

The `close` system call releases a file descriptor, making it free for reuse by a future `open`, `pipe`, or `dup` system call (see below). A newly allocated file descriptor is always the lowest-numbered unused descriptor of the current process.

File descriptors and `fork` interact to make I/O redirection easy to implement. `fork` copies the parent's file descriptor table along with its memory, so that the child starts with exactly the same open files as the parent. The system call `exec` replaces the calling process's memory but preserves its file table. This behavior allows the shell to implement *I/O redirection* by forking, re-opening chosen file descriptors in the child, and then calling `exec` to run the new program. Here is a simplified version of the code a shell runs for the command `cat < input.txt`:

```

char *argv[2];

argv[0] = "cat";
argv[1] = 0;
if(fork() == 0) {
    close(0);
    open("input.txt", O_RDONLY);
    exec("cat", argv);
}

```

After the child closes file descriptor 0, `open` is guaranteed to use that file descriptor for the newly opened `input.txt`: 0 will be the smallest available file descriptor. `cat` then executes with file descriptor 0 (standard input) referring to `input.txt`. The parent process's file descriptors are not changed by this sequence, since it modifies only the child's descriptors.

The code for I/O redirection in the xv6 shell works in exactly this way (`user/sh.c:83`). Recall that at this point in the code the shell has already forked the child shell and that `runcmd` will call `exec` to load the new program.

The second argument to `open` consists of a set of flags, expressed as bits, that control what `open` does. The possible values are defined in the file control (`fcntl`) header (`kernel/fcntl.h:1-5`): `O_RDONLY`, `O_WRONLY`, `O_RDWR`, `O_CREATE`, and `O_TRUNC`, which instruct `open` to open the file

for reading, or for writing, or for both reading and writing, to create the file if it doesn't exist, and to truncate the file to zero length.

Now it should be clear why it is helpful that `fork` and `exec` are separate calls: between the two, the shell has a chance to redirect the child's I/O without disturbing the I/O setup of the main shell. One could instead imagine a hypothetical combined `forkexec` system call, but the options for doing I/O redirection with such a call seem awkward. The shell could modify its own I/O setup before calling `forkexec` (and then un-do those modifications); or `forkexec` could take instructions for I/O redirection as arguments; or (least attractively) every program like `cat` could be taught to do its own I/O redirection.

Although `fork` copies the file descriptor table, each underlying file offset is shared between parent and child. Consider this example:

```
if(fork() == 0) {
    write(1, "hello ", 6);
    exit(0);
} else {
    wait(0);
    write(1, "world\n", 6);
}
```

At the end of this fragment, the file attached to file descriptor 1 will contain the data `hello world`. The `write` in the parent (which, thanks to `wait`, runs only after the child is done) picks up where the child's `write` left off. This behavior helps produce sequential output from sequences of shell commands, like `(echo hello; echo world) >output.txt`.

The `dup` system call duplicates an existing file descriptor, returning a new one that refers to the same underlying I/O object. Both file descriptors share an offset, just as the file descriptors duplicated by `fork` do. This is another way to write `hello world` into a file:

```
fd = dup(1);
write(1, "hello ", 6);
write(fd, "world\n", 6);
```

Two file descriptors share an offset if they were derived from the same original file descriptor by a sequence of `fork` and `dup` calls. Otherwise file descriptors do not share offsets, even if they resulted from `open` calls for the same file. `dup` allows shells to implement commands like this: `ls existing-file non-existing-file > tmp1 2>&1`. The `2>&1` tells the shell to give the command a file descriptor 2 that is a duplicate of descriptor 1. Both the name of the existing file and the error message for the non-existing file will show up in the file `tmp1`. The `xv6` shell doesn't support I/O redirection for the error file descriptor, but now you know how to implement it.

File descriptors are a powerful abstraction, because they hide the details of what they are connected to: a process writing to file descriptor 1 may be writing to a file, to a device like the console, or to a pipe.

## Chapter 2

# Operating system organization

A key requirement for an operating system is to support several activities at once. For example, using the system call interface described in Chapter 1 a process can start new processes with `fork`. The operating system must *time-share* the resources of the computer among these processes. For example, even if there are more processes than there are hardware CPUs, the operating system must ensure that all of the processes get a chance to execute. The operating system must also arrange for *isolation* between the processes. That is, if one process has a bug and malfunctions, it shouldn't affect processes that don't depend on the buggy process. Complete isolation, however, is too strong, since it should be possible for processes to intentionally interact; pipelines are an example. Thus an operating system must fulfill three requirements: multiplexing, isolation, and interaction.

This chapter provides an overview of how operating systems are organized to achieve these three requirements. It turns out there are many ways to do so, but this text focuses on mainstream designs centered around a *monolithic kernel*, which is used by many Unix operating systems. This chapter also provides an overview of an xv6 process, which is the unit of isolation in xv6, and the creation of the first process when xv6 starts.

Xv6 runs on a *multi-core*<sup>1</sup> RISC-V microprocessor, and much of its low-level functionality (for example, its process implementation) is specific to RISC-V. RISC-V is a 64-bit CPU, and xv6 is written in “LP64” C, which means long (L) and pointers (P) in the C programming language are 64 bits, but an `int` is 32 bits. This book assumes the reader has done a bit of machine-level programming on some architecture, and will introduce RISC-V-specific ideas as they come up. The user-level ISA [2] and privileged architecture [3] documents are the complete specifications. You may also refer to “The RISC-V Reader: An Open Architecture Atlas” [15].

The CPU in a complete computer is surrounded by support hardware, much of it in the form of I/O interfaces. Xv6 is written for the support hardware simulated by qemu's “-machine virt” option. This includes RAM, a ROM containing boot code, a serial connection to the user's keyboard/screen, and a disk for storage.

---

<sup>1</sup>By “multi-core” this text means multiple CPUs that share memory but execute in parallel, each with its own set of registers. This text sometimes uses the term *multiprocessor* as a synonym for multi-core, though multiprocessor can also refer more specifically to a computer with several distinct processor chips.

## 2.1 Abstracting physical resources

The first question one might ask when encountering an operating system is why have it at all? That is, one could implement the system calls in Figure 1.2 as a library, with which applications link. In this plan, each application could even have its own library tailored to its needs. Applications could directly interact with hardware resources and use those resources in the best way for the application (e.g., to achieve high or predictable performance). Some operating systems for embedded devices or real-time systems are organized in this way.

The downside of this library approach is that, if there is more than one application running, the applications must be well-behaved. For example, each application must periodically give up the CPU so that other applications can run. Such a *cooperative* time-sharing scheme may be OK if all applications trust each other and have no bugs. It's more typical for applications to not trust each other, and to have bugs, so one often wants stronger isolation than a cooperative scheme provides.

To achieve strong isolation it's helpful to forbid applications from directly accessing sensitive hardware resources, and instead to abstract the resources into services. For example, Unix applications interact with storage only through the file system's `open`, `read`, `write`, and `close` system calls, instead of reading and writing the disk directly. This provides the application with the convenience of pathnames, and it allows the operating system (as the implementer of the interface) to manage the disk. Even if isolation is not a concern, programs that interact intentionally (or just wish to keep out of each other's way) are likely to find a file system a more convenient abstraction than direct use of the disk.

Similarly, Unix transparently switches hardware CPUs among processes, saving and restoring register state as necessary, so that applications don't have to be aware of time-sharing. This transparency allows the operating system to share CPUs even if some applications are in infinite loops.

As another example, Unix processes use `exec` to build up their memory image, instead of directly interacting with physical memory. This allows the operating system to decide where to place a process in memory; if memory is tight, the operating system might even store some of a process's data on disk. `exec` also provides users with the convenience of a file system to store executable program images.

Many forms of interaction among Unix processes occur via file descriptors. Not only do file descriptors abstract away many details (e.g., where data in a pipe or file is stored), they are also defined in a way that simplifies interaction. For example, if one application in a pipeline fails, the kernel generates an end-of-file signal for the next process in the pipeline.

The system-call interface in Figure 1.2 is carefully designed to provide both programmer convenience and the possibility of strong isolation. The Unix interface is not the only way to abstract resources, but it has proved to be a good one.

## 2.2 User mode, supervisor mode, and system calls

Strong isolation requires a hard boundary between applications and the operating system. If the application makes a mistake, we don't want the operating system to fail or other applications to

fail. Instead, the operating system should be able to clean up the failed application and continue running other applications. To achieve strong isolation, the operating system must arrange that applications cannot modify (or even read) the operating system's data structures and instructions and that applications cannot access other processes' memory.

CPUs provide hardware support for strong isolation. For example, RISC-V has three modes in which the CPU can execute instructions: *machine mode*, *supervisor mode*, and *user mode*. Instructions executing in machine mode have full privilege; a CPU starts in machine mode. Machine mode is mostly intended for setting up the computer during boot. Xv6 executes a few lines in machine mode and then changes to supervisor mode.

In supervisor mode the CPU is allowed to execute *privileged instructions*: for example, enabling and disabling interrupts, reading and writing the register that holds the address of a page table, etc. If an application in user mode attempts to execute a privileged instruction, then the CPU doesn't execute the instruction, but switches to supervisor mode so that supervisor-mode code can terminate the application, because it did something it shouldn't be doing. Figure 1.1 in Chapter 1 illustrates this organization. An application can execute only user-mode instructions (e.g., adding numbers, etc.) and is said to be running in *user space*, while the software in supervisor mode can also execute privileged instructions and is said to be running in *kernel space*. The software running in kernel space (or in supervisor mode) is called the *kernel*.

An application that wants to invoke a kernel function (e.g., the `read` system call in xv6) must transition to the kernel; an application *cannot* invoke a kernel function directly. CPUs provide a special instruction that switches the CPU from user mode to supervisor mode and enters the kernel at an entry point specified by the kernel. (RISC-V provides the `ecall` instruction for this purpose.) Once the CPU has switched to supervisor mode, the kernel can then validate the arguments of the system call (e.g., check if the address passed to the system call is part of the application's memory), decide whether the application is allowed to perform the requested operation (e.g., check if the application is allowed to write the specified file), and then deny it or execute it. It is important that the kernel control the entry point for transitions to supervisor mode; if the application could decide the kernel entry point, a malicious application could, for example, enter the kernel at a point where the validation of arguments is skipped.

## 2.3 Kernel organization

A key design question is what part of the operating system should run in supervisor mode. One possibility is that the entire operating system resides in the kernel, so that the implementations of all system calls run in supervisor mode. This organization is called a *monolithic kernel*.

In this organization the entire operating system consists of a single program running with full hardware privilege. This organization is convenient because the OS designer doesn't have to decide which parts of the operating system don't need full hardware privilege. Furthermore, it is easier for different parts of the operating system to cooperate. For example, an operating system might have a buffer cache that can be shared both by the file system and the virtual memory system.

A downside of the monolithic organization is that the interactions among different parts of the operating system are often complex (as we will see in the rest of this text), and therefore it



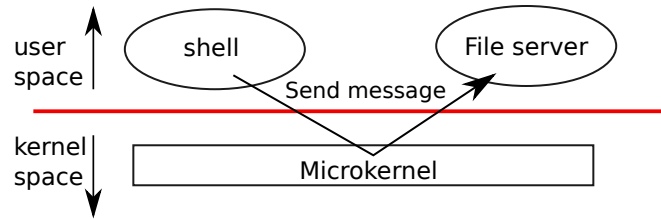


Figure 2.1: A microkernel with a file-system server

is easy for an operating system developer to make a mistake. In a monolithic kernel, a mistake is fatal, because an error in supervisor mode will often cause the kernel to fail. If the kernel fails, the computer stops working, and thus all applications fail too. The computer must reboot to start again.

To reduce the risk of mistakes in the kernel, OS designers can minimize the amount of operating system code that runs in supervisor mode, and execute the bulk of the operating system in user mode. This kernel organization is called a *microkernel*.

Figure 2.1 illustrates this microkernel design. In the figure, the file system runs as a user-level process. OS services running as processes are called servers. To allow applications to interact with the file server, the kernel provides an inter-process communication mechanism to send messages from one user-mode process to another. For example, if an application like the shell wants to read or write a file, it sends a message to the file server and waits for a response.

In a microkernel, the kernel interface consists of a few low-level functions for starting applications, sending messages, accessing device hardware, etc. This organization allows the kernel to be relatively simple, as most of the operating system resides in user-level servers.

In the real world, both monolithic kernels and microkernels are popular. Many Unix kernels are monolithic. For example, Linux has a monolithic kernel, although some OS functions run as user-level servers (e.g., the window system). Linux delivers high performance to OS-intensive applications, partially because the subsystems of the kernel can be tightly integrated.

Operating systems such as Minix, L4, and QNX are organized as a microkernel with servers, and have seen wide deployment in embedded settings. A variant of L4, seL4, is small enough that it has been verified for memory safety and other security properties [8].

There is much debate among developers of operating systems about which organization is better, and there is no conclusive evidence one way or the other. Furthermore, it depends much on what “better” means: faster performance, smaller code size, reliability of the kernel, reliability of the complete operating system (including user-level services), etc.

There are also practical considerations that may be more important than the question of which organization. Some operating systems have a microkernel but run some of the user-level services in kernel space for performance reasons. Some operating systems have monolithic kernels because that is how they started and there is little incentive to move to a pure microkernel organization, because new features may be more important than rewriting the existing operating system to fit a microkernel design.

From this book’s perspective, microkernel and monolithic operating systems share many key ideas. They implement system calls, they use page tables, they handle interrupts, they support

| File          | Description                                            |
|---------------|--------------------------------------------------------|
| bio.c         | Disk block cache for the file system.                  |
| console.c     | Connect to the user keyboard and screen.               |
| entry.S       | Very first boot instructions.                          |
| exec.c        | exec() system call.                                    |
| file.c        | File descriptor support.                               |
| fs.c          | File system.                                           |
| kalloc.c      | Physical page allocator.                               |
| kernelvec.S   | Handle traps from kernel.                              |
| log.c         | File system logging and crash recovery.                |
| main.c        | Control initialization of other modules during boot.   |
| pipe.c        | Pipes.                                                 |
| plic.c        | RISC-V interrupt controller.                           |
| printf.c      | Formatted output to the console.                       |
| proc.c        | Processes and scheduling.                              |
| sleeplock.c   | Locks that yield the CPU.                              |
| spinlock.c    | Locks that don't yield the CPU.                        |
| start.c       | Early machine-mode boot code.                          |
| string.c      | C string and byte-array library.                       |
| swtch.S       | Thread switching.                                      |
| syscall.c     | Dispatch system calls to handling function.            |
| sysfile.c     | File-related system calls.                             |
| sysproc.c     | Process-related system calls.                          |
| trampoline.S  | Assembly code to switch between user and kernel.       |
| trap.c        | C code to handle and return from traps and interrupts. |
| uart.c        | Serial-port console device driver.                     |
| virtio_disk.c | Disk device driver.                                    |
| vm.c          | Manage page tables and address spaces.                 |

Figure 2.2: Xv6 kernel source files.

processes, they use locks for concurrency control, they implement a file system, etc. This book focuses on these core ideas.

Xv6 is implemented as a monolithic kernel, like most Unix operating systems. Thus, the xv6 kernel interface corresponds to the operating system interface, and the kernel implements the complete operating system. Since xv6 doesn't provide many services, its kernel is smaller than some microkernels, but conceptually xv6 is monolithic.

## 2.4 Code: xv6 organization

The xv6 kernel source is in the `kernel/` sub-directory. The source is divided into files, following a rough notion of modularity; Figure 2.2 lists the files. The inter-module interfaces are defined in

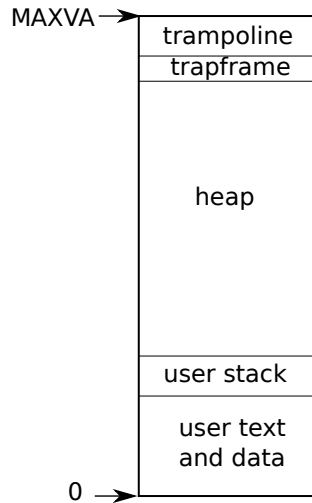


Figure 2.3: Layout of a process's virtual address space

`defs.h` (kernel/defs.h).

## 2.5 Process overview

The unit of isolation in xv6 (as in other Unix operating systems) is a *process*. The process abstraction prevents one process from wrecking or spying on another process's memory, CPU, file descriptors, etc. It also prevents a process from wrecking the kernel itself, so that a process can't subvert the kernel's isolation mechanisms. The kernel must implement the process abstraction with care because a buggy or malicious application may trick the kernel or hardware into doing something bad (e.g., circumventing isolation). The mechanisms used by the kernel to implement processes include the user/supervisor mode flag, address spaces, and time-slicing of threads.

To help enforce isolation, the process abstraction provides the illusion to a program that it has its own private machine. A process provides a program with what appears to be a private memory system, or *address space*, which other processes cannot read or write. A process also provides the program with what appears to be its own CPU to execute the program's instructions.

Xv6 uses page tables (which are implemented by hardware) to give each process its own address space. The RISC-V page table translates (or “maps”) a *virtual address* (the address that an RISC-V instruction manipulates) to a *physical address* (an address that the CPU sends to main memory).

Xv6 maintains a separate page table for each process that defines that process's address space. As illustrated in Figure 2.3, an address space includes the process's *user memory* starting at virtual address zero. Instructions come first, followed by global variables, then the stack, and finally a “heap” area (for malloc) that the process can expand as needed. There are a number of factors that limit the maximum size of a process's address space: pointers on the RISC-V are 64 bits wide; the hardware uses only the low 39 bits when looking up virtual addresses in page tables; and xv6 uses only 38 of those 39 bits. Thus, the maximum address is  $2^{38} - 1 = 0x3ffffffffff$ , which

is `MAXVA` (`kernel/riscv.h:378`). At the top of the address space `xv6` places a *trampoline* page (4096 bytes) and a *trapframe* page. `Xv6` uses these two pages to transition into the kernel and back; the trampoline page contains the code to transition in and out of the kernel, and the trapframe is where the kernel saves the process's user registers, as Chapter 4 explains.

The `xv6` kernel maintains many pieces of state for each process, which it gathers into a `struct proc` (`kernel/proc.h:85`). A process's most important pieces of kernel state are its page table, its kernel stack, and its run state. We'll use the notation `p->xxx` to refer to elements of the `proc` structure; for example, `p->pagetable` is a pointer to the process's page table.

Each process has a thread of control (or *thread* for short) that holds the state needed to execute the process. At any given time, a thread might be executing on a CPU, or suspended (not executing, but capable of resuming executing in the future). To switch a CPU between processes, the kernel suspends the thread currently running on that CPU and saves its state, and restores the state of another process's previously-suspended thread. Much of the state of a thread (local variables, function call return addresses) is stored on the thread's stacks. Each process has two stacks: a user stack and a kernel stack (`p->kstack`). When the process is executing user instructions, only its user stack is in use, and its kernel stack is empty. When the process enters the kernel (for a system call or interrupt), the kernel code executes on the process's kernel stack; while a process is in the kernel, its user stack still contains saved data, but isn't actively used. A process's thread alternates between actively using its user stack and its kernel stack. The kernel stack is separate (and protected from user code) so that the kernel can execute even if a process has wrecked its user stack.

A process can make a system call by executing the RISC-V `ecall` instruction. This instruction raises the hardware privilege level and changes the program counter to a kernel-defined entry point. The code at the entry point switches to the process's kernel stack and executes the kernel instructions that implement the system call. When the system call completes, the kernel switches back to the user stack and returns to user space by calling the `sret` instruction, which lowers the hardware privilege level and resumes executing user instructions just after the system call instruction. A process's thread can "block" in the kernel to wait for I/O, and resume where it left off when the I/O has finished.

`p->state` indicates whether the process is allocated, ready to run, currently running on a CPU, waiting for I/O, or exiting.

`p->pagetable` holds the process's page table, in the format that the RISC-V hardware expects. `Xv6` causes the paging hardware to use a process's `p->pagetable` when executing that process in user space. A process's page table also serves as the record of the addresses of the physical pages allocated to store the process's memory.

In summary, a process bundles two design ideas: an address space to give a process the illusion of its own memory, and a thread to give the process the illusion of its own CPU. In `xv6`, a process consists of one address space and one thread. In real operating systems a process may have more than one thread to take advantage of multiple CPUs.

## 2.6 Code: starting xv6, the first process and system call

To make xv6 more concrete, we'll outline how the kernel starts and runs the first process. The subsequent chapters will describe the mechanisms that show up in this overview in more detail.

When the RISC-V computer powers on, it initializes itself and runs a boot loader which is stored in read-only memory. The boot loader loads the xv6 kernel into memory. Then, in machine mode, the CPU executes xv6 starting at `_entry` (`kernel/entry.S:7`). The RISC-V starts with paging hardware disabled: virtual addresses map directly to physical addresses.

The loader loads the xv6 kernel into memory at physical address `0x80000000`. The reason it places the kernel at `0x80000000` rather than `0x0` is because the address range `0x0 : 0x80000000` contains I/O devices.

The instructions at `_entry` set up a stack so that xv6 can run C code. Xv6 declares space for an initial stack, `stack0`, in the file `start.c` (`kernel/start.c:11`). The code at `_entry` loads the stack pointer register `sp` with the address `stack0+4096`, the top of the stack, because the stack on RISC-V grows down. Now that the kernel has a stack, `_entry` calls into C code at `start` (`kernel/start.c:15`).

The function `start` performs some configuration that is only allowed in machine mode, and then switches to supervisor mode. To enter supervisor mode, RISC-V provides the instruction `mret`. This instruction is most often used to return from a previous call from supervisor mode to machine mode. `start` isn't returning from such a call, but sets things up as if it were: it sets the previous privilege mode to supervisor in the register `mstatus`, it sets the return address to `main` by writing `main`'s address into the register `mepc`, disables virtual address translation in supervisor mode by writing `0` into the page-table register `satp`, and delegates all interrupts and exceptions to supervisor mode.

Before jumping into supervisor mode, `start` performs one more task: it programs the clock chip to generate timer interrupts. With this housekeeping out of the way, `start` "returns" to supervisor mode by calling `mret`. This causes the program counter to change to `main` (`kernel/main.c:11`), the address previously stored in `mepc`.

After `main` (`kernel/main.c:11`) initializes several devices and subsystems, it creates the first process by calling `userinit` (`kernel/proc.c:233`). The first process executes a small program written in RISC-V assembly, which makes the first system call in xv6. `initcode.S` (`user/initcode.S:3`) loads the number for the `exec` system call, `SYS_EXEC` (`kernel/syscall.h:8`), into register `a7`, and then calls `ecall` to re-enter the kernel.

The kernel uses the number in register `a7` in `syscall` (`kernel/syscall.c:132`) to call the desired system call. The system call table (`kernel/syscall.c:107`) maps `SYS_EXEC` to the function `sys_exec`, which the kernel invokes. As we saw in Chapter 1, `exec` replaces the memory and registers of the current process with a new program (in this case, `/init`).

Once the kernel has completed `exec`, it returns to user space in the `/init` process. `init` (`user/init.c:15`) creates a new console device file if needed and then opens it as file descriptors `0`, `1`, and `2`. Then it starts a shell on the console. The system is up.

# Chapter 3

## Page tables

Page tables are the most popular mechanism through which the operating system provides each process with its own private address space and memory. Page tables determine what memory addresses mean, and what parts of physical memory can be accessed. They allow xv6 to isolate different process's address spaces and to multiplex them onto a single physical memory. Page tables are a popular design because they provide a level of indirection that allow operating systems to perform many tricks. Xv6 performs a few tricks: mapping the same memory (a trampoline page) in several address spaces, and guarding kernel and user stacks with an unmapped page. The rest of this chapter explains the page tables that the RISC-V hardware provides and how xv6 uses them.

### 3.1 Paging hardware

As a reminder, RISC-V instructions (both user and kernel) manipulate virtual addresses. The machine's RAM, or physical memory, is indexed with physical addresses. The RISC-V page table hardware connects these two kinds of addresses, by mapping each virtual address to a physical address.

Xv6 runs on Sv39 RISC-V, which means that only the bottom 39 bits of a 64-bit virtual address are used; the top 25 bits are not used. In this Sv39 configuration, a RISC-V page table is logically an array of  $2^{27}$  (134,217,728) *page table entries* (PTEs). Each PTE contains a 44-bit physical page number (PPN) and some flags. The paging hardware translates a virtual address by using the top 27 bits of the 39 bits to index into the page table to find a PTE, and making a 56-bit physical address whose top 44 bits come from the PPN in the PTE and whose bottom 12 bits are copied from the original virtual address. Figure 3.1 shows this process with a logical view of the page table as a simple array of PTEs (see Figure 3.2 for a fuller story). A page table gives the operating system control over virtual-to-physical address translations at the granularity of aligned chunks of  $4096$  ( $2^{12}$ ) bytes. Such a chunk is called a *page*.

In Sv39 RISC-V, the top 25 bits of a virtual address are not used for translation. The physical address also has room for growth: there is room in the PTE format for the physical page number to grow by another 10 bits. The designers of RISC-V chose these numbers based on technology predictions.  $2^{39}$  bytes is 512 GB, which should be enough address space for applications running

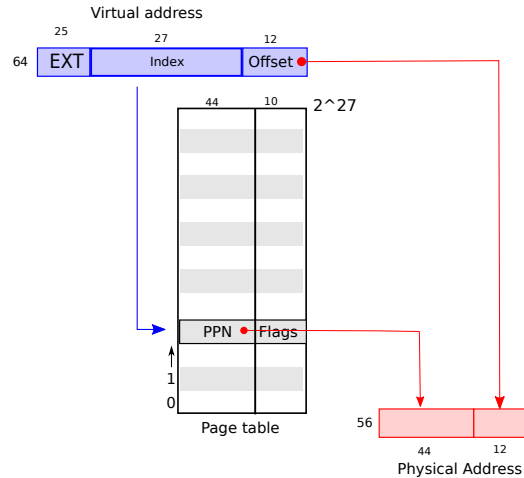


Figure 3.1: RISC-V virtual and physical addresses, with a simplified logical page table.

on RISC-V computers.  $2^{56}$  is enough physical memory space for the near future to fit many I/O devices and RAM chips. If more is needed, the RISC-V designers have defined Sv48 with 48-bit virtual addresses [3].

As Figure 3.2 shows, a RISC-V CPU translates a virtual address into a physical in three steps. A page table is stored in physical memory as a three-level tree. The root of the tree is a 4096-byte page-table page that contains 512 PTEs, which contain the physical addresses for page-table pages in the next level of the tree. Each of those pages contains 512 PTEs for the final level in the tree. The paging hardware uses the top 9 bits of the 27 bits to select a PTE in the root page-table page, the middle 9 bits to select a PTE in a page-table page in the next level of the tree, and the bottom 9 bits to select the final PTE. (In Sv48 RISC-V a page table has four levels, and bits 39 through 47 of a virtual address index into the top-level.)

If any of the three PTEs required to translate an address is not present, the paging hardware raises a *page-fault exception*, leaving it up to the kernel to handle the exception (see Chapter 4).

The three-level structure of Figure 3.2 allows a memory-efficient way of recording PTEs, compared to the single-level design of Figure 3.1. In the common case in which large ranges of virtual addresses have no mappings, the three-level structure can omit entire page directories. For example, if an application uses only a few pages starting at address zero, then the entries 1 through 511 of the top-level page directory are invalid, and the kernel doesn't have to allocate pages those for 511 intermediate page directories. Furthermore, the kernel also doesn't have to allocate pages for the bottom-level page directories for those 511 intermediate page directories. So, in this example, the three-level design saves 511 pages for intermediate page directories and  $511 \times 512$  pages for bottom-level page directories.

Although a CPU walks the three-level structure in hardware as part of executing a load or store instruction, a potential downside of three levels is that the CPU must load three PTEs from memory to perform the translation of the virtual address in the load/store instruction to a physical address. To avoid the cost of loading PTEs from physical memory, a RISC-V CPU caches page table entries in a *Translation Look-aside Buffer (TLB)*.

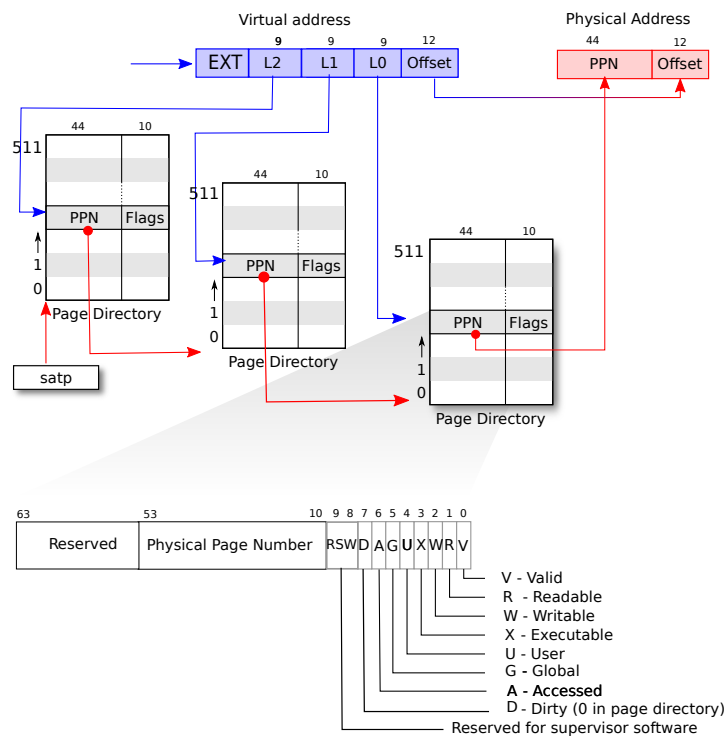


Figure 3.2: RISC-V address translation details.

Each PTE contains flag bits that tell the paging hardware how the associated virtual address is allowed to be used. `PTE_V` indicates whether the PTE is present: if it is not set, a reference to the page causes an exception (i.e., is not allowed). `PTE_R` controls whether instructions are allowed to read to the page. `PTE_W` controls whether instructions are allowed to write to the page. `PTE_X` controls whether the CPU may interpret the content of the page as instructions and execute them. `PTE_U` controls whether instructions in user mode are allowed to access the page; if `PTE_U` is not set, the PTE can be used only in supervisor mode. Figure 3.2 shows how it all works. The flags and all other page hardware-related structures are defined in (`kernel/riscv.h`)

To tell a CPU to use a page table, the kernel must write the physical address of the root page-table page into the `satp` register. A CPU will translate all addresses generated by subsequent instructions using the page table pointed to by its own `satp`. Each CPU has its own `satp` so that different CPUs can run different processes, each with a private address space described by its own page table.

From the kernel’s point of view, a page table is data stored in memory, and the kernel creates and modifies page tables using code much like you might see for any tree-shaped data structure.

A few notes about terms used in this book. *Physical memory* refers to storage cells in RAM. A byte of physical memory has an address, called a *physical address*. Instructions that dereference addresses (such as loads, stores, jumps, and function calls) use only virtual addresses, which the paging hardware translates to physical addresses, and then sends to the RAM hardware to read or write storage. An *address space* is the set of virtual addresses that are valid in a given page table;



each xv6 process has a separate user address space, and the xv6 kernel has its own address space as well. *User memory* refers to a process’s user address space plus the physical memory that the page table allows the process to access. *Virtual memory* refers to the ideas and techniques associated with managing page tables and using them to achieve goals such as isolation.

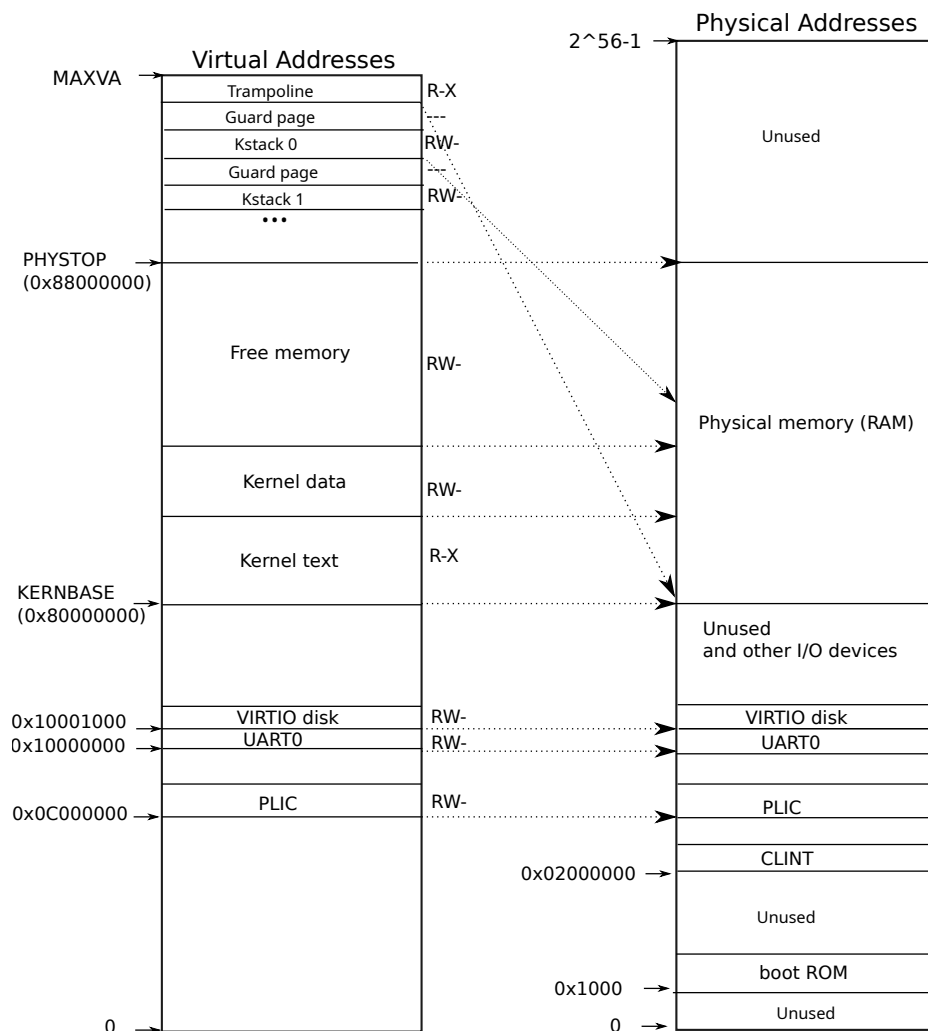


Figure 3.3: On the left, xv6’s kernel address space. RWX refer to PTE read, write, and execute permissions. On the right, the RISC-V physical address space that xv6 expects to see.

## 3.2 Kernel address space

Xv6 maintains one page table per process, describing each process’s user address space, plus a single page table that describes the kernel’s address space. The kernel configures the layout of its address space to give itself access to physical memory and various hardware resources at predictable

virtual addresses. Figure 3.3 shows how this layout maps kernel virtual addresses to physical addresses. The file (`kernel/memlayout.h`) declares the constants for xv6’s kernel memory layout.

QEMU simulates a computer that includes RAM (physical memory) starting at physical address `0x80000000` and continuing through at least `0x88000000`, which xv6 calls `PHYSTOP`. The QEMU simulation also includes I/O devices such as a disk interface. QEMU exposes the device interfaces to software as *memory-mapped* control registers that sit below `0x80000000` in the physical address space. The kernel can interact with the devices by reading/writing these special physical addresses; such reads and writes communicate with the device hardware rather than with RAM. Chapter 4 explains how xv6 interacts with devices.

The kernel gets at RAM and memory-mapped device registers using “direct mapping,” that is, mapping the resources at virtual addresses that are equal to the physical address. For example, the kernel itself is located at `KERNBASE=0x80000000` in both the virtual address space and in physical memory. Direct mapping simplifies kernel code that reads or writes physical memory. For example, when `fork` allocates user memory for the child process, the allocator returns the physical address of that memory; `fork` uses that address directly as a virtual address when it is copying the parent’s user memory to the child.

There are a couple of kernel virtual addresses that aren’t direct-mapped:

- The trampoline page. It is mapped at the top of the virtual address space; user page tables have this same mapping. Chapter 4 discusses the role of the trampoline page, but we see here an interesting use case of page tables; a physical page (holding the trampoline code) is mapped twice in the virtual address space of the kernel: once at top of the virtual address space and once with a direct mapping.
- The kernel stack pages. Each process has its own kernel stack, which is mapped high so that below it xv6 can leave an unmapped *guard page*. The guard page’s PTE is invalid (i.e., `PTE_V` is not set), so that if the kernel overflows a kernel stack, it will likely cause an exception and the kernel will panic. Without a guard page an overflowing stack would overwrite other kernel memory, resulting in incorrect operation. A panic crash is preferable.

While the kernel uses its stacks via the high-memory mappings, they are also accessible to the kernel through a direct-mapped address. An alternate design might have just the direct mapping, and use the stacks at the direct-mapped address. In that arrangement, however, providing guard pages would involve unmapping virtual addresses that would otherwise refer to physical memory, which would then be hard to use.

The kernel maps the pages for the trampoline and the kernel text with the permissions `PTE_R` and `PTE_X`. The kernel reads and executes instructions from these pages. The kernel maps the other pages with the permissions `PTE_R` and `PTE_W`, so that it can read and write the memory in those pages. The mappings for the guard pages are invalid.

### 3.3 Code: creating an address space

Most of the xv6 code for manipulating address spaces and page tables resides in `vm.c` (`kernel/vm.c:1`). The central data structure is `pagetable_t`, which is really a pointer to a RISC-V

root page-table page; a `pagetable_t` may be either the kernel page table, or one of the per-process page tables. The central functions are `walk`, which finds the PTE for a virtual address, and `mappages`, which installs PTEs for new mappings. Functions starting with `kvm` manipulate the kernel page table; functions starting with `uvm` manipulate a user page table; other functions are used for both. `copyout` and `copyin` copy data to and from user virtual addresses provided as system call arguments; they are in `vm.c` because they need to explicitly translate those addresses in order to find the corresponding physical memory.

Early in the boot sequence, `main` calls `kvminit` (kernel/vm.c:54) to create the kernel's page table using `kvmmake` (kernel/vm.c:20). This call occurs before xv6 has enabled paging on the RISC-V, so addresses refer directly to physical memory. `kvmmake` first allocates a page of physical memory to hold the root page-table page. Then it calls `kvmmap` to install the translations that the kernel needs. The translations include the kernel's instructions and data, physical memory up to `PHYSTOP`, and memory ranges which are actually devices. `proc_mapstacks` (kernel/proc.c:33) allocates a kernel stack for each process. It calls `kvmmap` to map each stack at the virtual address generated by `KSTACK`, which leaves room for the invalid stack-guard pages.

`kvmmap` (kernel/vm.c:132) calls `mappages` (kernel/vm.c:144), which installs mappings into a page table for a range of virtual addresses to a corresponding range of physical addresses. It does this separately for each virtual address in the range, at page intervals. For each virtual address to be mapped, `mappages` calls `walk` to find the address of the PTE for that address. It then initializes the PTE to hold the relevant physical page number, the desired permissions (`PTE_W`, `PTE_X`, and/or `PTE_R`), and `PTE_V` to mark the PTE as valid (kernel/vm.c:165).

`walk` (kernel/vm.c:86) mimics the RISC-V paging hardware as it looks up the PTE for a virtual address (see Figure 3.2). `walk` descends the page table one level at a time, using each level's 9 bits of virtual address to index into the relevant page directory page. At each level it finds either the PTE of the next level's page directory page, or the PTE of final page (kernel/vm.c:92). If a PTE in a first or second level page directory page isn't valid, then the required directory page hasn't yet been allocated; if the `alloc` argument is set, `walk` allocates a new page-table page and puts its physical address in the PTE. It returns the address of the PTE in the lowest layer in the tree (kernel/vm.c:102).

The above code depends on physical memory being direct-mapped into the kernel virtual address space. For example, as `walk` descends levels of the page table, it pulls the (physical) address of the next-level-down page table from a PTE (kernel/vm.c:94), and then uses that address as a virtual address to fetch the PTE at the next level down (kernel/vm.c:92).

`main` calls `kvminithart` (kernel/vm.c:62) to install the kernel page table. It writes the physical address of the root page-table page into the register `satp`. After this the CPU will translate addresses using the kernel page table. Since the kernel uses a direct mapping, the now virtual address of the next instruction will map to the right physical memory address.

Each RISC-V CPU caches page table entries in a *Translation Look-aside Buffer (TLB)*, and when xv6 changes a page table, it must tell the CPU to invalidate corresponding cached TLB entries. If it didn't, then at some point later the TLB might use an old cached mapping, pointing to a physical page that in the meantime has been allocated to another process, and as a result, a process might be able to scribble on some other process's memory. The RISC-V has an

instruction `sfence.vma` that flushes the current CPU's TLB. Xv6 executes `sfence.vma` in `kvminithart` after reloading the `satp` register, and in the trampoline code that switches to a user page table before returning to user space (`kernel/trampoline.S:89`).

It is also necessary to issue `sfence.vma` before changing `satp`, in order to wait for completion of all outstanding loads and stores. This wait ensures that preceding updates to the page table have completed, and ensures that preceding loads and stores use the old page table, not the new one.

To avoid flushing the complete TLB, RISC-V CPUs may support address space identifiers (ASIDs) [3]. The kernel can then flush just the TLB entries for a particular address space. Xv6 does not use this feature.

## 3.4 Physical memory allocation

The kernel must allocate and free physical memory at run-time for page tables, user memory, kernel stacks, and pipe buffers.

Xv6 uses the physical memory between the end of the kernel and `PHYSTOP` for run-time allocation. It allocates and frees whole 4096-byte pages at a time. It keeps track of which pages are free by threading a linked list through the pages themselves. Allocation consists of removing a page from the linked list; freeing consists of adding the freed page to the list.

## 3.5 Code: Physical memory allocator

The allocator resides in `kalloc.c` (`kernel/kalloc.c:1`). The allocator's data structure is a *free list* of physical memory pages that are available for allocation. Each free page's list element is a `struct run` (`kernel/kalloc.c:17`). Where does the allocator get the memory to hold that data structure? It store each free page's `run` structure in the free page itself, since there's nothing else stored there. The free list is protected by a spin lock (`kernel/kalloc.c:21-24`). The list and the lock are wrapped in a struct to make clear that the lock protects the fields in the struct. For now, ignore the lock and the calls to `acquire` and `release`; Chapter 6 will examine locking in detail.

The function `main` calls `kinit` to initialize the allocator (`kernel/kalloc.c:27`). `kinit` initializes the free list to hold every page between the end of the kernel and `PHYSTOP`. Xv6 ought to determine how much physical memory is available by parsing configuration information provided by the hardware. Instead xv6 assumes that the machine has 128 megabytes of RAM. `kinit` calls `freerange` to add memory to the free list via per-page calls to `kfree`. A PTE can only refer to a physical address that is aligned on a 4096-byte boundary (is a multiple of 4096), so `freerange` uses `PGROUNDUP` to ensure that it frees only aligned physical addresses. The allocator starts with no memory; these calls to `kfree` give it some to manage.

The allocator sometimes treats addresses as integers in order to perform arithmetic on them (e.g., traversing all pages in `freerange`), and sometimes uses addresses as pointers to read and write memory (e.g., manipulating the `run` structure stored in each page); this dual use of addresses is the main reason that the allocator code is full of C type casts.

The function `kfree` (`kernel/kalloc.c:47`) begins by setting every byte in the memory being freed to the value 1. This will cause code that uses memory after freeing it (uses “dangling references”) to read garbage instead of the old valid contents; hopefully that will cause such code to break faster. Then `kfree` prepends the page to the free list: it casts `pa` to a pointer to `struct run`, records the old start of the free list in `r->next`, and sets the free list equal to `r`. `kalloc` removes and returns the first element in the free list.

## 3.6 Process address space

Each process has its own page table, and when `xv6` switches between processes, it also changes page tables. Figure 3.4 shows a process’s address space in more detail than Figure 2.3. A process’s user memory starts at virtual address zero and can grow up to `MAXVA` (`kernel/riscv.h:375`), allowing a process to address in principle 256 Gigabytes of memory.

A process’s address space consists of pages that contain the text of the program (which `xv6` maps with the permissions `PTE_R`, `PTE_X`, and `PTE_U`), pages that contain the pre-initialized data of the program, a page for the stack, and pages for the heap. `Xv6` maps the data, stack, and heap with the permissions `PTE_R`, `PTE_W`, and `PTE_U`.

Using permissions within a user address space is a common technique to harden a user process. If the text were mapped with `PTE_W`, then a process could accidentally modify its own program; for example, a programming error may cause the program to write to a null pointer, modifying instructions at address 0, and then continue running, perhaps creating more havoc. To detect such errors immediately, `xv6` maps the text without `PTE_W`; if a program accidentally attempts to store to address 0, the hardware will refuse to execute the store and raises a page fault (see Section 4.6). The kernel then kills the process and prints out an informative message so that the developer can track down the problem.

Similarly, by mapping data without `PTE_X`, a user program cannot accidentally jump to an address in the program’s data and start executing at that address.

In the real world, hardening a process by setting permissions carefully also aids in defending against security attacks. An adversary may feed carefully-constructed input to a program (e.g., a Web server) that triggers a bug in the program in the hope of turning that bug into an exploit [14]. Setting permissions carefully and other techniques, such as randomizing of the layout of the user address space, make such attacks harder.

The stack is a single page, and is shown with the initial contents as created by `exec`. Strings containing the command-line arguments, as well as an array of pointers to them, are at the very top of the stack. Just under that are values that allow a program to start at `main` as if the function `main(argc, argv)` had just been called.

To detect a user stack overflowing the allocated stack memory, `xv6` places an inaccessible guard page right below the stack by clearing the `PTE_U` flag. If the user stack overflows and the process tries to use an address below the stack, the hardware will generate a page-fault exception because the guard page is inaccessible to a program running in user mode. A real-world operating system might instead automatically allocate more memory for the user stack when it overflows.

When a process asks `xv6` for more user memory, `xv6` grows the process’s heap. `Xv6` first uses

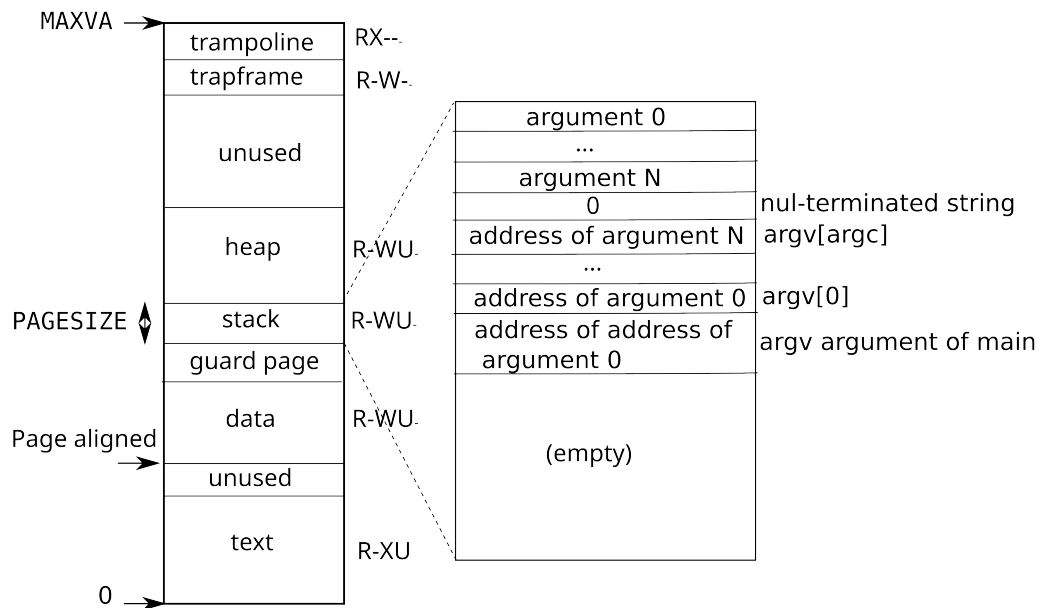


Figure 3.4: A process's user address space, with its initial stack.

`kalloc` to allocate physical pages. It then adds PTEs to the process's page table that point to the new physical pages. Xv6 sets the `PTE_W`, `PTE_R`, `PTE_U`, and `PTE_V` flags in these PTEs. Most processes do not use the entire user address space; xv6 leaves `PTE_V` clear in unused PTEs.

We see here a few nice examples of use of page tables. First, different processes' page tables translate user addresses to different pages of physical memory, so that each process has private user memory. Second, each process sees its memory as having contiguous virtual addresses starting at zero, while the process's physical memory can be non-contiguous. Third, the kernel maps a page with trampoline code at the top of the user address space (without `PTE_U`), thus a single page of physical memory shows up in all address spaces, but can be used only by the kernel.

### 3.7 Code: `sbrk`

`sbrk` is the system call for a process to shrink or grow its memory. The system call is implemented by the function `growproc` (`kernel/proc.c:260`). `growproc` calls `uvmalloc` or `uvmdealloc`, depending on whether `n` is positive or negative. `uvmalloc` (`kernel/vm.c:233`) allocates physical memory with `kalloc`, zeros the allocated memory, and adds PTEs to the user page table with `mappages`. `uvmdealloc` calls `uvmunmap` (`kernel/vm.c:178`), which uses `walk` to find PTEs and `kfree` to free the physical memory they refer to.

Xv6 uses a process's page table not just to tell the hardware how to map user virtual addresses,

but also as the only record of which physical memory pages are allocated to that process. That is the reason why freeing user memory (in `uvmunmap`) requires examination of the user page table.

### 3.8 Code: `exec`

`exec` is a system call that replaces a process's user address space with data read from a file, called a binary or executable file. A binary is typically the output of the compiler and linker, and holds machine instructions and program data. `exec` (`kernel/exec.c:23`) opens the named binary `path` using `namei` (`kernel/exec.c:36`), which is explained in Chapter 8. Then, it reads the ELF header. Xv6 binaries are formatted in the widely-used *ELF format*, defined in (`kernel/elf.h`). An ELF binary consists of an ELF header, `struct elfhdr` (`kernel/elf.h:6`), followed by a sequence of program section headers, `struct proghdr` (`kernel/elf.h:25`). Each `proghdr` describes a section of the application that must be loaded into memory; xv6 programs have two program section headers: one for instructions and one for data.

The first step is a quick check that the file probably contains an ELF binary. An ELF binary starts with the four-byte “magic number” `0x7F`, `'E'`, `'L'`, `'F'`, or `ELF_MAGIC` (`kernel/elf.h:3`). If the ELF header has the right magic number, `exec` assumes that the binary is well-formed.

`exec` allocates a new page table with no user mappings with `proc_pagetable` (`kernel/exec.c:49`), allocates memory for each ELF segment with `uvmalloc` (`kernel/exec.c:65`), and loads each segment into memory with `loadseg` (`kernel/exec.c:10`). `loadseg` uses `walkaddr` to find the physical address of the allocated memory at which to write each page of the ELF segment, and `readi` to read from the file.

The program section header for `/init`, the first user program created with `exec`, looks like this:

```
# objdump -p user/_init

user/_init:      file format elf64-little

Program Header:
0x70000003 off    0x00000000000006bb0 vaddr 0x0000000000000000
                                paddr 0x0000000000000000 align 2**0
      filesz 0x000000000000004a memsz 0x0000000000000000 flags r--
LOAD off      0x00000000000001000 vaddr 0x0000000000000000
                                paddr 0x0000000000000000 align 2**12
      filesz 0x00000000000001000 memsz 0x00000000000001000 flags r-x
LOAD off      0x00000000000002000 vaddr 0x00000000000001000
                                paddr 0x00000000000001000 align 2**12
      filesz 0x0000000000000010 memsz 0x0000000000000030 flags rw-
STACK off     0x00000000000000000 vaddr 0x0000000000000000
                                paddr 0x0000000000000000 align 2**4
      filesz 0x0000000000000000 memsz 0x0000000000000000 flags rw-
```

We see that the text segment should be loaded at virtual address `0x0` in memory (without write permissions) from content at offset `0x1000` in the file. We also see that the data should be loaded at address `0x1000`, which is at a page boundary, and without executable permissions.

A program section header's `filesz` may be less than the `memsz`, indicating that the gap between them should be filled with zeroes (for C global variables) rather than read from the file. For `/init`, the data `filesz` is 0x10 bytes and `memsz` is 0x30 bytes, and thus `uvmmalloc` allocates enough physical memory to hold 0x30 bytes, but reads only 0x10 bytes from the file `/init`.

Now `exec` allocates and initializes the user stack. It allocates just one stack page. `exec` copies the argument strings to the top of the stack one at a time, recording the pointers to them in `ustack`. It places a null pointer at the end of what will be the `argv` list passed to `main`. The values for `argc` and `argv` are passed to `main` through the system-call return path: `argc` is passed via the system call return value, which goes in `a0`, and `argv` is passed through the `a1` entry of the process's `trapframe`.

`exec` places an inaccessible page just below the stack page, so that programs that try to use more than one page will fault. This inaccessible page also allows `exec` to deal with arguments that are too large; in that situation, the `copyout` (`kernel/vm.c:359`) function that `exec` uses to copy arguments to the stack will notice that the destination page is not accessible, and will return -1.

During the preparation of the new memory image, if `exec` detects an error like an invalid program segment, it jumps to the label `bad`, frees the new image, and returns -1. `exec` must wait to free the old image until it is sure that the system call will succeed: if the old image is gone, the system call cannot return -1 to it. The only error cases in `exec` happen during the creation of the image. Once the image is complete, `exec` can commit to the new page table (`kernel/exec.c:125`) and free the old one (`kernel/exec.c:129`).

`exec` loads bytes from the ELF file into memory at addresses specified by the ELF file. Users or processes can place whatever addresses they want into an ELF file. Thus `exec` is risky, because the addresses in the ELF file may refer to the kernel, accidentally or on purpose. The consequences for an unwary kernel could range from a crash to a malicious subversion of the kernel's isolation mechanisms (i.e., a security exploit). Xv6 performs a number of checks to avoid these risks. For example `if(ph.vaddr + ph.memsz < ph.vaddr)` checks for whether the sum overflows a 64-bit integer. The danger is that a user could construct an ELF binary with a `ph.vaddr` that points to a user-chosen address, and `ph.memsz` large enough that the sum overflows to 0x1000, which will look like a valid value. In an older version of xv6 in which the user address space also contained the kernel (but not readable/writable in user mode), the user could choose an address that corresponded to kernel memory and would thus copy data from the ELF binary into the kernel. In the RISC-V version of xv6 this cannot happen, because the kernel has its own separate page table; `loadseg` loads into the process's page table, not in the kernel's page table.

It is easy for a kernel developer to omit a crucial check, and real-world kernels have a long history of missing checks whose absence can be exploited by user programs to obtain kernel privileges. It is likely that xv6 doesn't do a complete job of validating user-level data supplied to the kernel, which a malicious user program might be able to exploit to circumvent xv6's isolation.

## 3.9 Real world

Like most operating systems, xv6 uses the paging hardware for memory protection and mapping. Most operating systems make far more sophisticated use of paging than xv6 by combining paging



and page-fault exceptions, which we will discuss in Chapter 4.

Xv6 is simplified by the kernel’s use of a direct map between virtual and physical addresses, and by its assumption that there is physical RAM at address 0x80000000, where the kernel expects to be loaded. This works with QEMU, but on real hardware it turns out to be a bad idea; real hardware places RAM and devices at unpredictable physical addresses, so that (for example) there might be no RAM at 0x80000000, where xv6 expect to be able to store the kernel. More serious kernel designs exploit the page table to turn arbitrary hardware physical memory layouts into predictable kernel virtual address layouts.

RISC-V supports protection at the level of physical addresses, but xv6 doesn’t use that feature.

On machines with lots of memory it might make sense to use RISC-V’s support for “super pages.” Small pages make sense when physical memory is small, to allow allocation and page-out to disk with fine granularity. For example, if a program uses only 8 kilobytes of memory, giving it a whole 4-megabyte super-page of physical memory is wasteful. Larger pages make sense on machines with lots of RAM, and may reduce overhead for page-table manipulation.

The xv6 kernel’s lack of a `malloc`-like allocator that can provide memory for small objects prevents the kernel from using sophisticated data structures that would require dynamic allocation. A more elaborate kernel would likely allocate many different sizes of small blocks, rather than (as in xv6) just 4096-byte blocks; a real kernel allocator would need to handle small allocations as well as large ones.

Memory allocation is a perennial hot topic, the basic problems being efficient use of limited memory and preparing for unknown future requests [9]. Today people care more about speed than space efficiency.

## 3.10 Exercises

1. Parse RISC-V’s device tree to find the amount of physical memory the computer has.
2. Write a user program that grows its address space by one byte by calling `sbrk(1)`. Run the program and investigate the page table for the program before the call to `sbrk` and after the call to `sbrk`. How much space has the kernel allocated? What does the PTE for the new memory contain?
3. Modify xv6 to use super pages for the kernel.
4. Unix implementations of `exec` traditionally include special handling for shell scripts. If the file to execute begins with the text `#!`, then the first line is taken to be a program to run to interpret the file. For example, if `exec` is called to run `myprog arg1` and `myprog`’s first line is `#!/interp`, then `exec` runs `/interp` with command line `/interp myprog arg1`. Implement support for this convention in xv6.
5. Implement address space layout randomization for the kernel.

# Chapter 4

## Traps and system calls

There are three kinds of event which cause the CPU to set aside ordinary execution of instructions and force a transfer of control to special code that handles the event. One situation is a system call, when a user program executes the `ecall` instruction to ask the kernel to do something for it. Another situation is an *exception*: an instruction (user or kernel) does something illegal, such as divide by zero or use an invalid virtual address. The third situation is a device *interrupt*, when a device signals that it needs attention, for example when the disk hardware finishes a read or write request.

This book uses *trap* as a generic term for these situations. Typically whatever code was executing at the time of the trap will later need to resume, and shouldn't need to be aware that anything special happened. That is, we often want traps to be transparent; this is particularly important for device interrupts, which the interrupted code typically doesn't expect. The usual sequence is that a trap forces a transfer of control into the kernel; the kernel saves registers and other state so that execution can be resumed; the kernel executes appropriate handler code (e.g., a system call implementation or device driver); the kernel restores the saved state and returns from the trap; and the original code resumes where it left off.

Xv6 handles all traps in the kernel; traps are not delivered to user code. Handling traps in the kernel is natural for system calls. It makes sense for interrupts since isolation demands that only the kernel be allowed to use devices, and because the kernel is a convenient mechanism with which to share devices among multiple processes. It also makes sense for exceptions since xv6 responds to all exceptions from user space by killing the offending program.

Xv6 trap handling proceeds in four stages: hardware actions taken by the RISC-V CPU, some assembly instructions that prepare the way for kernel C code, a C function that decides what to do with the trap, and the system call or device-driver service routine. While commonality among the three trap types suggests that a kernel could handle all traps with a single code path, it turns out to be convenient to have separate code for two distinct cases: traps from user space, and traps from kernel space. Kernel code (assembler or C) that processes a trap is often called a *handler*; the first handler instructions are usually written in assembler (rather than C) and are sometimes called a *vector*.

## 4.1 RISC-V trap machinery

Each RISC-V CPU has a set of control registers that the kernel writes to tell the CPU how to handle traps, and that the kernel can read to find out about a trap that has occurred. The RISC-V documents contain the full story [3]. `riscv.h` (kernel/riscv.h:1) contains definitions that xv6 uses. Here's an outline of the most important registers:

- `stvec`: The kernel writes the address of its trap handler here; the RISC-V jumps to the address in `stvec` to handle a trap.
- `sepc`: When a trap occurs, RISC-V saves the program counter here (since the `pc` is then overwritten with the value in `stvec`). The `sret` (return from trap) instruction copies `sepc` to the `pc`. The kernel can write `sepc` to control where `sret` goes.
- `scause`: RISC-V puts a number here that describes the reason for the trap.
- `sscratch`: The trap handler code uses `sscratch` to help it avoid overwriting user registers before saving them.
- `sstatus`: The SIE bit in `sstatus` controls whether device interrupts are enabled. If the kernel clears SIE, the RISC-V will defer device interrupts until the kernel sets SIE. The SPP bit indicates whether a trap came from user mode or supervisor mode, and controls to what mode `sret` returns.

The above registers relate to traps handled in supervisor mode, and they cannot be read or written in user mode.

Each CPU on a multi-core chip has its own set of these registers, and more than one CPU may be handling a trap at any given time.

When it needs to force a trap, the RISC-V hardware does the following for all trap types:

1. If the trap is a device interrupt, and the `sstatus` SIE bit is clear, don't do any of the following.
2. Disable interrupts by clearing the SIE bit in `sstatus`.
3. Copy the `pc` to `sepc`.
4. Save the current mode (user or supervisor) in the SPP bit in `sstatus`.
5. Set `scause` to reflect the trap's cause.
6. Set the mode to supervisor.
7. Copy `stvec` to the `pc`.
8. Start executing at the new `pc`.

Note that the CPU doesn't switch to the kernel page table, doesn't switch to a stack in the kernel, and doesn't save any registers other than the `pc`. Kernel software must perform these tasks. One reason that the CPU does minimal work during a trap is to provide flexibility to software; for example, some operating systems omit a page table switch in some situations to increase trap performance.

It's worth thinking about whether any of the steps listed above could be omitted, perhaps in search of faster traps. Though there are situations in which a simpler sequence can work, many of the steps would be dangerous to omit in general. For example, suppose that the CPU didn't switch program counters. Then a trap from user space could switch to supervisor mode while still running user instructions. Those user instructions could break user/kernel isolation, for example by modifying the `satp` register to point to a page table that allowed accessing all of physical memory. It is thus important that the CPU switch to a kernel-specified instruction address, namely `stvec`.

## 4.2 Traps from user space

Xv6 handles traps differently depending on whether the trap occurs while executing in the kernel or in user code. Here is the story for traps from user code; Section 4.5 describes traps from kernel code.

A trap may occur while executing in user space if the user program makes a system call (`ecall` instruction), or does something illegal, or if a device interrupts. The high-level path of a trap from user space is `uservec` (`kernel/trampoline.S:22`), then `usertrap` (`kernel/trap.c:37`); and when returning, `usertrapret` (`kernel/trap.c:90`) and then `userret` (`kernel/trampoline.S:101`).

A major constraint on the design of xv6's trap handling is the fact that the RISC-V hardware does not switch page tables when it forces a trap. This means that the trap handler address in `stvec` must have a valid mapping in the user page table, since that's the page table in force when the trap handling code starts executing. Furthermore, xv6's trap handling code needs to switch to the kernel page table; in order to be able to continue executing after that switch, the kernel page table must also have a mapping for the handler pointed to by `stvec`.

Xv6 satisfies these requirements using a *trampoline* page. The trampoline page contains `uservec`, the xv6 trap handling code that `stvec` points to. The trampoline page is mapped in every process's page table at address `TRAMPOLINE`, which is at the top of the virtual address space so that it will be above memory that programs use for themselves. The trampoline page is also mapped at address `TRAMPOLINE` in the kernel page table. See Figure 2.3 and Figure 3.3. Because the trampoline page is mapped in the user page table, traps can start executing there in supervisor mode. Because the trampoline page is mapped at the same address in the kernel address space, the trap handler can continue to execute after it switches to the kernel page table.

The code for the `uservec` trap handler is in `trampoline.S` (`kernel/trampoline.S:22`). When `uservec` starts, all 32 registers contain values owned by the interrupted user code. These 32 values need to be saved somewhere in memory, so that later on the kernel can restore them before returning to user space. Storing to memory requires use of a register to hold the address, but at this point there are no general-purpose registers available! Luckily RISC-V provides a helping hand in the form of the `sscratch` register. The `csrw` instruction at the start of `uservec` saves `a0` in

`sscratch`. Now `uservec` has one register (`a0`) to play with.

`uservec`'s next task is to save the 32 user registers. The kernel allocates, for each process, a page of memory for a `trapframe` structure that (among other things) has space to save the 32 user registers (`kernel/proc.h:43`). Because `satp` still refers to the user page table, `uservec` needs the `trapframe` to be mapped in the user address space. Xv6 maps each process's `trapframe` at virtual address `TRAPFRAME` in that process's user page table; `TRAPFRAME` is just below `TRAMPOLINE`. The process's `p->trapframe` also points to the `trapframe`, though at its physical address so the kernel can use it through the kernel page table.

Thus `uservec` loads address `TRAPFRAME` into `a0` and saves all the user registers there, including the user's `a0`, read back from `sscratch`.

The `trapframe` contains the address of the current process's kernel stack, the current CPU's `hartid`, the address of the `usertrap` function, and the address of the kernel page table. `uservec` retrieves these values, switches `satp` to the kernel page table, and jumps to `usertrap`.

The job of `usertrap` is to determine the cause of the trap, process it, and return (`kernel/trap.c:37`). It first changes `stvec` so that a trap while in the kernel will be handled by `kernelvec` rather than `uservec`. It saves the `sepc` register (the saved user program counter), because `usertrap` might call `yield` to switch to another process's kernel thread, and that process might return to user space, in the process of which it will modify `sepc`. If the trap is a system call, `usertrap` calls `syscall` to handle it; if a device interrupt, `devintr`; otherwise it's an exception, and the kernel kills the faulting process. The system call path adds four to the saved user program counter because RISC-V, in the case of a system call, leaves the program pointer pointing to the `ecall` instruction but user code needs to resume executing at the subsequent instruction. On the way out, `usertrap` checks if the process has been killed or should yield the CPU (if this trap is a timer interrupt).

The first step in returning to user space is the call to `usertrapret` (`kernel/trap.c:90`). This function sets up the RISC-V control registers to prepare for a future trap from user space: setting `stvec` to `uservec` and preparing the `trapframe` fields that `uservec` relies on. `usertrapret` sets `sepc` to the previously saved user program counter. At the end, `usertrapret` calls `userret` on the trampoline page that is mapped in both user and kernel page tables; the reason is that assembly code in `userret` will switch page tables.

`usertrapret`'s call to `userret` passes a pointer to the process's user page table in `a0` (`kernel/trampoline.S:101`). `userret` switches `satp` to the process's user page table. Recall that the user page table maps both the trampoline page and `TRAPFRAME`, but nothing else from the kernel. The trampoline page mapping at the same virtual address in user and kernel page tables allows `userret` to keep executing after changing `satp`. From this point on, the only data `userret` can use is the register contents and the content of the `trapframe`. `userret` loads the `TRAPFRAME` address into `a0`, restores saved user registers from the `trapframe` via `a0`, restores the saved user `a0`, and executes `sret` to return to user space.

## 4.3 Code: Calling system calls

Chapter 2 ended with `initcode.S` invoking the `exec` system call (`user/initcode.S:11`). Let's look at how the user call makes its way to the `exec` system call's implementation in the kernel.

`initcode.S` places the arguments for `exec` in registers `a0` and `a1`, and puts the system call number in `a7`. System call numbers match the entries in the `syscalls` array, a table of function pointers (`kernel/syscall.c:107`). The `ecall` instruction traps into the kernel and causes `uservec`, `usertrap`, and then `syscall` to execute, as we saw above.

`syscall` (`kernel/syscall.c:132`) retrieves the system call number from the saved `a7` in the `trapframe` and uses it to index into `syscalls`. For the first system call, `a7` contains `SYS_exec` (`kernel/syscall.h:8`), resulting in a call to the system call implementation function `sys_exec`.

When `sys_exec` returns, `syscall` records its return value in `p->trapframe->a0`. This will cause the original user-space call to `exec()` to return that value, since the C calling convention on RISC-V places return values in `a0`. System calls conventionally return negative numbers to indicate errors, and zero or positive numbers for success. If the system call number is invalid, `syscall` prints an error and returns `-1`.

## 4.4 Code: System call arguments

System call implementations in the kernel need to find the arguments passed by user code. Because user code calls system call wrapper functions, the arguments are initially where the RISC-V C calling convention places them: in registers. The kernel trap code saves user registers to the current process's trap frame, where kernel code can find them. The kernel functions `argint`, `argaddr`, and `argfd` retrieve the *n*'th system call argument from the trap frame as an integer, pointer, or a file descriptor. They all call `argraw` to retrieve the appropriate saved user register (`kernel/syscall.c:34`).

Some system calls pass pointers as arguments, and the kernel must use those pointers to read or write user memory. The `exec` system call, for example, passes the kernel an array of pointers referring to string arguments in user space. These pointers pose two challenges. First, the user program may be buggy or malicious, and may pass the kernel an invalid pointer or a pointer intended to trick the kernel into accessing kernel memory instead of user memory. Second, the xv6 kernel page table mappings are not the same as the user page table mappings, so the kernel cannot use ordinary instructions to load or store from user-supplied addresses.

The kernel implements functions that safely transfer data to and from user-supplied addresses. `fetchstr` is an example (`kernel/syscall.c:25`). File system calls such as `exec` use `fetchstr` to retrieve string file-name arguments from user space. `fetchstr` calls `copyinstr` to do the hard work.

`copyinstr` (`kernel/vm.c:415`) copies up to `max` bytes to `dst` from virtual address `srcva` in the user page table `pagetable`. Since `pagetable` is *not* the current page table, `copyinstr` uses `walkaddr` (which calls `walk`) to look up `srcva` in `pagetable`, yielding physical address `pa0`. The kernel's page table maps all of physical RAM at virtual addresses that are equal to the RAM's physical address. This allows `copyinstr` to directly copy string bytes from `pa0` to `dst`. `walkaddr` (`kernel/vm.c:109`) checks that the user-supplied virtual address is part of the process's

user address space, so programs cannot trick the kernel into reading other memory. A similar function, `copyout`, copies data from the kernel to a user-supplied address.

## 4.5 Traps from kernel space

Xv6 handles traps from kernel code in a different way than traps from user code. When entering the kernel, `usertrap` points `stvec` to the assembly code at `kernelvec` (`kernel/kernelvec.S:12`). Since `kernelvec` only executes if xv6 was already in the kernel, `kernelvec` can rely on `satp` being set to the kernel page table, and on the stack pointer referring to a valid kernel stack. `kernelvec` pushes all 32 registers onto the stack, from which it will later restore them so that the interrupted kernel code can resume without disturbance.

`kernelvec` saves the registers on the stack of the interrupted kernel thread, which makes sense because the register values belong to that thread. This is particularly important if the trap causes a switch to a different thread – in that case the trap will actually return from the stack of the new thread, leaving the interrupted thread’s saved registers safely on its stack.

`kernelvec` jumps to `kerneltrap` (`kernel/trap.c:135`) after saving registers. `kerneltrap` is prepared for two types of traps: device interrupts and exceptions. It calls `devintr` (`kernel/trap.c:185`) to check for and handle the former. If the trap isn’t a device interrupt, it must be an exception, and that is always a fatal error if it occurs in the xv6 kernel; the kernel calls `panic` and stops executing.

If `kerneltrap` was called due to a timer interrupt, and a process’s kernel thread is running (as opposed to a scheduler thread), `kerneltrap` calls `yield` to give other threads a chance to run. At some point one of those threads will yield, and let our thread and its `kerneltrap` resume again. Chapter 7 explains what happens in `yield`.

When `kerneltrap`’s work is done, it needs to return to whatever code was interrupted by the trap. Because a `yield` may have disturbed `sepc` and the previous mode in `sstatus`, `kerneltrap` saves them when it starts. It now restores those control registers and returns to `kernelvec` (`kernel/kernelvec.S:38`). `kernelvec` pops the saved registers from the stack and executes `sret`, which copies `sepc` to `pc` and resumes the interrupted kernel code.

It’s worth thinking through how the trap return happens if `kerneltrap` called `yield` due to a timer interrupt.

Xv6 sets a CPU’s `stvec` to `kernelvec` when that CPU enters the kernel from user space; you can see this in `usertrap` (`kernel/trap.c:29`). There’s a window of time when the kernel has started executing but `stvec` is still set to `uservec`, and it’s crucial that no device interrupt occur during that window. Luckily the RISC-V always disables interrupts when it starts to take a trap, and `usertrap` doesn’t enable them again until after it sets `stvec`.

## 4.6 Page-fault exceptions

Xv6’s response to exceptions is quite boring: if an exception happens in user space, the kernel kills the faulting process. If an exception happens in the kernel, the kernel panics. Real operating